LangChain4j

EMSI - Université Côte d'Azur
Richard Grin
Version 4.4 – 28/10/26

1

Plan du support

- LangChain
- Présentation et bases de LangChain4j
- AiServices
- Extraction des données
- Outils
- Modération
- Guardrails
- Observabilité
- Streaming
- APIs et outils autour de l'IA
- Références

2

LangChain

3

Présentation

- https://www.langchain.com/
- Framework open source pour développer des applications qui utilisent des LMs
- Couche abstraite pour travailler avec l'API de nombreux LMs (OpenAI, Gemini, Llama, Anthropic, Mistral, ...)
- Permet de combiner des LMs avec des applications et des sources de données externes
- Permet de créer des « chaînes », des séries d'actions enchaînées les unes aux autres
- Librairie officielle pour Python et JavaScript, pas pour Java

4

Applications IA

- Les tâches complexes nécessitent
  - un découpage en plusieurs sous-tâches
  - l'utilisation de plusieurs types de modèles
  - leur configuration
  - l'utilisation de plusieurs types de supports (textes, images, sons,...), de ressources externes et d'outils
- LangChain facilite l'exécution de ces tâches et leur indépendance par rapport aux produits utilisés

5

Présentation et bases de LangChain4j

- Modèles
- Templates
- Mémoire
- Logging
- Modèles embeddings

6

Présentation

- https://langchain4j.github.io/langchain4j/
- Au départ, transposition de LangChain à Java, mais il s'en est progressivement différencié pour une intégration plus naturelle à Java
- Standard de facto pour IA avec Java

7

Ce qui est offert

- APIs unifiées pour les LMs, les BD vectorielles (pour embeddings)
- Boite à outils pour prompts, gestion de la mémoire, traitement des ressources externes, RAG, « outils » des LMs, systèmes agentiques
- Intégration avec les principaux frameworks Java (Jakarta EE, Spring Boot, Quarkus, Micronaut, Helidon)
- Documentation très complète avec de nombreux exemples de code pour des cas d'utilisation

8

Contenu API

- 2 niveaux d'abstraction
  - Bas niveau pour manipuler les modèles, les messages, les embeddings, ...
  - Plus haut niveau en utilisant `AiServices` (configuration déclarative)

9

Intégration avec les LMs

- Anthropic (texte, image)
- Azure OpenAi
- Google AI Gemini (texte, image, audio, vidéo, PDF)
- HuggingFace
- Jlama
- LocalAI
- Mistral AI (texte, image)
- Ollama
- OpenAI (avec texte, image, audio, PDF)
- ...

10

Dépendances Maven - Gemini

```
<dependency>
    <groupId>dev.langchain4j</groupId>
    <artifactId>langchain4j</artifactId>
    <version>1.22.0</version>
</dependency>
<dependency>
    <groupId>dev.langchain4j</groupId>
    <artifactId>langchain4j-google-genai</artifactId>
    <version>1.22.0</version>
    <type>jar</type>
</dependency>
```



11

Dépendances Maven - OpenAI

```
<dependency>
    <groupId>dev.langchain4j</groupId>
    <artifactId>langchain4j</artifactId>
    <version>1.22.0</version>
</dependency>
<dependency>
    <groupId>dev.langchain4j</groupId>
    <artifactId>langchain4j-open-ai</artifactId>
    <version>1.22.0</version>
</dependency>
```


12

Clé secrète pour API

- La plupart des LMs nécessitent une clé pour identifier l'utilisateur
- Il n'est pas bon d'écrire la valeur de la clé dans le code
- Le plus simple est d'ajouter une variable d'environnement dans l'OS et de récupérer sa valeur dans le code ; par exemple `String cle = System.getenv("CLE_GEMINI");`

Types de modèles

- Interface `ChatModel` : représente un LM qui traite une requête et retourne une réponse
- Interface `StreamingChatModel` : comme `ChatModel` mais la réponse est retournée en streaming (token par token)
- Interface `EmbeddingModel` : pour convertir des textes en embeddings
- Interface `ImageModel` : générateur d'image à partir d'un texte
- Interface `ModerationModel` : surveillance des contenus échangés avec les LMs

Classe d'un modèle concret

- LangChain4j contient une API pour chacun des LMs qu'il supporte
- Chaque API implémente les interfaces qui représentent les modèles ; par exemple, `GoogleGenAiChatModel` (Gemini) et `OpenAiResponsesChatModel` (OpenAI) implémentent `ChatModel`
- Une instance de la classe d'implémentation s'obtient et se configure le plus souvent avec le pattern « builder »

Pattern « builder »

- Modèle de conception pour créer et configurer des instances de classes :
  - configuration se fait en appelant des méthodes d'une classe « Builder »
  - méthode `build()` du Builder crée l'instance configurée
- LangChain4j l'utilise non seulement pour les LMs, mais aussi pour créer les instances de nombreuses classes : `ChatRequest`, `ChatRequestParameters`, `AiServices`, et bien d'autres

Exemple simple

```
String cle = System.getenv("GEMINI_KEY");
// Création du modèle avec un builder
ChatModel modele = GoogleGenAiChatModel.builder()
                .apiKey(cle)
                .modelName("gemini-3.5-flash-lite")
                .build();
// Pose une question au modèle
String reponse =
    modele.chat("Traduis 'bonjour' en anglais");
// Affiche réponse du modèle (hello)
System.out.println(reponse);
```


Interface `ChatModel`

- Package dev.langchain4j.model.chat
- Pour simple question - réponse : `String chat(String userMessage)`
- 3 méthodes `chat` qui retourne une `ChatResponse` :
  - `chat(List<ChatMessage> messages)`
  - `chat(ChatMessage... messages)`
  - `chat(ChatRequest requête)`

ChatRequest, ChatResponse

- `ChatRequest` : Requête envoyée au LM ; peut contenir des messages (`ChatMessage`), le nom du modèle, les hyperparamètres du LM à utiliser (`ChatRequestParameters`, température, …), informations sur les outils disponibles, format de la réponse à utiliser (`ResponseFormat`), …

  Pourquoi plusieurs messages ?
- `ChatResponse` : réponse du LM, avec des informations complémentaires : nom du modèle, métadonnées (`Metadata`), tokens utilisés (`TokenUsage`), cause de l'arrêt de la réponse (`FinishReason`)



Méthodes chat de ChatModel

- `ChatResponse chat(ChatRequest requête)` : permet d'interroger le LM en utilisant les paramètres, les messages déjà dans requête et les informations (longueur max de la réponse, outils, …) de la requête
- `ChatResponse chat(List<ChatMessage> messages)` `ChatResponse chat(ChatMessage... messages)` : permet d'interroger le LM en utilisant les messages passés en paramètres
- `String chat(String userMessage)` : le message passé en paramètre et la réponse du LM sous la forme d'une simple `String`

ChatRequest

- La question posée au modèle peut être une `String` mais aussi une `ChatRequest` pour prendre en compte des cas plus complexes
- Contient (voir `Builder` pour le nom du getter)
  - des messages (`List<ChatMessage>`)
  - des paramètres (`ChatRequestParameters`)
  - des spécifications d'outils (`ToolSpecifications`)
  - un format de réponse (`ResponseFormat`)

Interface ChatResponse

- Package dev.langchain4j.model.chat.response
- Réponse du LM
- Méthodes :
- `AiMessage aiMessage()` (`aiMessage.text()` pour avoir le texte de la réponse)
- `ChatResponseMetadata metadata()`
- `TokenUsage tokenUsage()` : usage des tokens, nombres de tokens en entrée et en sortie
- `FinishReason finishReason()` : une raison de fin de réponse
- `String modelName()`

Enumération FinishReason

- Package dev.langchain4j.model.output
- Indique pourquoi la génération du LM s'est arrêtée
- `STOP` : le LM a décidé qu'il avait fini de traiter la requête
- `LENGTH` : limite atteinte pour le nombre de token
- `TOOL_EXECUTION` : signale qu'il faut appeler un outil
- `CONTENT_FILTER` : arrêt à cause d'un contenu jugé contraire à la politique de modération du LM
- `OTHER` : autre raison

Classe TokenUsage

- Package dev.langchain4j.model.output
- `Integer inputTokenCount()`
- `Integer outputTokenCount()`
- `Integer totalTokenCount()`
- Possible de cumuler 2 `TokenUsage`

Exemple utilisation

```
UserMessage message = UserMessage.from("...");
 reponse = modele.chat(message);

// Récupérer le texte de la réponse
String texteReponse = reponse.aiMessage().text();

// Obtenir le nombre de tokens utilisés
TokenUsage usage = reponse.tokenUsage();
int totalTokens = usage.totalTokenCount();
int inputTokens = usage.inputTokenCount();
int outputTokens = usage.outputTokenCount();
```


Metadata

- Interface du package dev.langchain4j.model.chat.response
- Représente les métadonnées habituelles supportées par les LMs ; chaque fournisseur de LM peut étendre cette interface
- Les métadonnées de base supportées (getters et méthodes du builder) : `finishReason`, `tokenUsage`, `modelName`

Classe ChatRequest.Builder

- Retourné par `ChatRequest.builder()`
- Contient des méthodes qui retournent `ChatRequest.Builder` :
  - `messages(ChatMessage... messages)` et `messages(List<ChatMessage> messages)`
  - `parameters(ChatRequestParameters parameters)`
  - `toolSpecifications(ToolSpecification... toolSpecifications)` et `toolSpecifications(List<ToolSpecification> toolSpecifications)`
  - Nombreuses autres méthodes…

Exemple avec ChatRequest

```
ChatModel modele =
    GoogleGenAiChatModel.builder()
      .apiKey(cle)
      .modelName("gemini-3.5-flash-lite")
      .temperature(0.7)
      .timeout(Duration.ofSeconds(60)) // si service surchargé
      .responseFormat(ResponseFormat.JSON) // juste pour voir
      .build();
ChatRequest requete = ChatRequest.builder()
      .temperature(0.5) // juste pour cette requête
      .messages(SystemMessage.from("Réponds en hésitant"),
                UserMessage.from("Superficie de la France ?"))
      .build();
ChatResponse reponse = modele.chat(requete);
System.out.println(reponse.aiMessage().text());
System.out.println(reponse.tokenUsage().totalTokenCount());
```


Exemple avec OpenAI

```
ChatModel modele =
    OpenAiResponsesChatModel.builder()
      .apiKey(cle)
      .modelName("gpt-6-luna")
//      .responseFormat("json_schema")
//      .strictJsonSchema(true)
      .build();
```


Interface ChatMessage

- Package dev.langchain4j.data.message
- Représente un message échangé pendant une conversation avec un LM :
  - une requête vers le LM
  - ou une réponse du LM
- Implémenté par 4 classes du même package

Types de ChatMessage

- `UserMessage` : message de l'application ou de l'utilisateur qui peut contenir du texte, des images, de l'audio, de la vidéo ou du PDF
- `SystemMessage` : message pour le système ; décrit le rôle du LM, comment il doit se comporter, le style de réponse,... Souvent au début de la conversation
- `AiMessage` : message généré par le LM en réponse à un `UserMessage` ; peut contenir un texte (`String`, que l'on obtient par la méthode `text()`) ou une requête pour exécuter un outil (`ToolExecutionRequest`)
- `ToolExecutionResultMessage` : envoyé au LM ; résultat d'une `ToolExecutionRequest`



Classe UserMessage

- Package dev.langchain4j.data.message
- Plusieurs façons de créer un `UserMessage` avec un texte (`String`), un ou plusieurs `Content` de différents types (image, audio, vidéo ou fichier PDF), et optionnellement un nom d'utilisateur :
  - constructeurs
  - méthode builder()
  - méthodes `static from`
  - méthodes `static userMessage` (comme `from`)
- Méthodes `String singleText()`, `List<Content> contents()` pour retrouver le contenu du message

Interface Content

- Package dev.langchain4j.data.message
- Pour donner un des contenus d'un message utilisateur
- Une seule méthode `ContentType type()`
- `ContentType` est une énumération dont les valeurs sont `TEXT`, `TEXT_FILE`, `PDF`, `AUDIO`, `IMAGE`, `VIDEO`
- Implémentée par `TextContent`, `TextFileContent`, `PdfFileContent`, `ImageContent`, `AudioContent`, `VideoContent`

Multimodalité

- Tous les modèles n'acceptent pas tous les types de contenu pour poser une question
- Gemini et OpenAI acceptent les types image (`ImageContent`), vidéo (`VideoContent`), audio (`AudioContent`), PDF (`PdfFileContent`) ; toutes ces classes implémentent l'interface `Content` (classes et interfaces dans package dev.langchain4j.data.message)
- Par exemple, on peut demander à Gemini de décrire une image (voir exemple à suivre)

Classe ImageContent

- Package dev.langchain4j.data.message
- Création avec un des constructeurs (choix parmi les paramètres de type `Image`, `String` ou `URI` pour l'URL, `String` pour l'image codée en base64, `String` pour le type Mime, `ImageContent.DetailLevel` pour niveau de détails de l'image) ou avec une des méthodes `static from` (avec les mêmes paramètres que les constructeurs)

Exemple d'utilisation d'image

```
reponse =
   model.chat(UserMessage.from(
       ImageContent.from("https://.../uneimage.jpg"),
       TextContent.from("Décris-moi cette image.")
   )
);
// Récupération du texte de la réponse
System.out.println(reponse.aiMessage().text());
```

Classe `Image`

- Package dev.langchain4j.data.image
- Représente une image par son URL ou encodée en Base64
- Utilisé pour la génération d'image à partir d'un texte
- Pattern builder pour la création, avec les paramètres de type `String` pour le contenu de l'image codé en Base64, `String` pour le type Mime, `String` ou `URI` pour l'URL

37

Classe `SystemMessage`

- Package dev.langchain4j.data.message
- Le plus souvent défini par le développeur, pas l'utilisateur
- Instructions sur le comportement, le style, le rôle du LM
- Les LMs et les frameworks qui les utilisent (comme LangChain4j) donnent le plus souvent la priorité aux messages système s'il y a contradiction avec des messages utilisateur

38

`AiMessage`

- Package dev.langchain4j.data.message
- Réponse du LM
- Peut contenir
  - du texte (obtenu avec la méthode `text()`)
  - une requête pour exécuter un ou plusieurs outils (`toolExecutionRequests()`)
  - attributs (dépendent de l'implémentation du modèle ; `attributes()`)
  - la chaîne de raisonnement interne du modèle (thinking()) ; retourne souvent null car le modèle n'expose pas sa chaîne de raisonnement

39

Exemples de messages

```
SystemMessage systemMessage =
    SystemMessage.from("Toutes tes réponses en majuscules");
UserMessage userMessage1 =
    UserMessage.from("Hello, mon nom est Julien");
AiMessage aiMessage1 =
    model.chat(systemMessage, userMessage1).aiMessage();
System.out.println(aiMessage1);
/* Affichage : { text = "BONJOUR JULIEN.", thinking = null,
toolExecutionRequests = [], attributes = {} } */
UserMessage userMessage2 = UserMessage.from("Mon nom ?");
String aiMessage2 =
    model.chat(userMessage1, aiMessage1, userMessage2)
         .aiMessage().text();
// "Votre nom est Julien."
```

Expliquez cette réponse

Pourquoi cette réponse n'est pas en majuscules ?

40

`ChatRequestParameters`

- Package dev.langchain4j.model.chat.request
- Regroupe les paramètres du LM : nom du modèle, hyperparamètres pour l'inférence (température, maxOutputTokens, description d'outils utilisables, …)
- Création avec le pattern builder
- Paramètres spécifiés à la création du `ChatModel` (considérés alors comme les valeurs par défaut) ou bien attachés à un `ChatRequest` et utilisés à l'appel de la méthode `chat`

41

DefaultChatRequestParameters

- Classe de dev.langchain4j.model.chat.request
- Implémentation par défaut de l'interface ChatRequestParameters

42

Exemple

```
ChatRequestParameters parameters =
    ChatRequestParameters.builder()
        .modelName("...")
        .temperature(0.7)
        .build();
// Utilisation à la création du modèle
ChatModel model = OpenAiResponsesChatModel.builder()
        .apiKey(System.getenv("OPENAI_KEY"))
        .defaultRequestParameters(parameters)
        .build();
// Ou bien attaché à une requête particulière
ChatRequest chatRequest = ChatRequest.builder()
        .messages(UserMessage.from("Hello"))
        .parameters(parameters)
        .build();
```


Response<T>

- Classe du package dev.langchain4j.model.output
- Représente une réponse pour un modèle d'embeddings, un modèle de modération ou la génération d'image

Quelles métadonnées ?

- Permet de récupérer des métadonnées sur la réponse
- `T` : type de contenu généré par le modèle
  - `Embedding` pour un `EmbeddingModel`
  - `Moderation` pour un `ModerationModel`
  - `Image` pour un `ImageModel`

Vous pouvez commencer les tests 1 et 2 du TP 2



Méthodes de Response<T>

- `T content()` : récupère le contenu
- `TokenUsage tokenUsage()` : récupère les statistiques sur l'usage du modèle (nombre de tokens en entrée et en sortie)
- `Map<String, Object> metadata()` : récupère les métadonnées
- `FinishReason finishReason()` : récupère la raison de l'arrêt de la génération par le LM

Classe PromptTemplate

- Package dev.langchain4j.model.input
- Pour obtenir des bonnes réponses à un LM, il faut souvent poser les questions avec un bon format ; un template permet d'imposer un format prédéfini
- `PromptTemplate` représente un template qui peut contenir des variables, par exemple `{{nom}}`

Méthodes de PromptTemplate

- `static PromptTemplate from(String template)` : création d'un template
- `Prompt apply(Map<String, Object> valeurs)` : donner des valeurs aux variables du template ; la map contient le nom (clé de la map) et la valeur pour chaque variable
- `Prompt apply(Object valeur)` : appliquer une valeur pour un template qui n'a qu'une seule variable **qui a nécessairement le nom « `it` »**

Exemple

```
PromptTemplate template = PromptTemplate.from("""
  Réponds en te basant sur l'information suivante :
  {{context}}
  Question : {{question}}""");

Prompt prompt = template.apply(Map.of(
    "context", "Les LLMs sont entraînés sur de grands
corpus de texte.",
    "question", "Comment fonctionne un LLM ?"
));

String reponse = modele.chat(prompt.text())
```

Variables prédéfinies

- `current_date` ; valeur `LocalDate.now()`
- `current_time` ; valeur `LocalTime.now()`
- `current_date_time` ; valeur `LocalDateTime.now()`

Exemple avec variable prédéfinie

```
Prompt prompt = PromptTemplate
    .from("Quel âge a-t-il en date du {{current_date}} ?")
    .apply(Map.of());
reponse = model.chat(prompt.text());
```


Exemples avec variables

- ```
  Prompt prompt = PromptTemplate
      .from("Quel âge a {{it}} en date du {{current_date}}?")
      .apply("Quentin Tarantino");
  ```
- ```
  Prompt prompt = PromptTemplate
      .from("Quel âge a {{nom}} en date du {{current_date}}?")
      .apply(Map.of("nom","Quentin Tarantino"));
  ```

Vous pouvez expliquer la différence ?

Requête few shot avec template

```
PromptTemplate promptTemplate =
    PromptTemplate.from("""
Analyse le sentiment du texte qui suit. Réponds avec un
seul mot qui décrit le sentiment. Voici des exemples :
  INPUT: C'est une bonne nouvelle !
  OUTPUT: POSITIF

  INPUT: Pi est à peu près égal à 3.14
  OUTPUT: NEUTRE

  INPUT: Cette pizza n'est pas bonne !
  OUTPUT: NEGATIF

  INPUT: {{it}}
  OUTPUT: """);
```



Classe `Prompt`

- Package dev.langchain4j.model.input
- Méthode `text()` transforme une instance en `String`
- Méthode `toUserMessage()` transforme une instance en `UserMessage`
- Méthode `toSystemMessage()` transforme une instance en `SystemMessage`
- Méthode `toAiMessage()` transforme une instance en `AiMessage`

`@StructuredPrompt`

- Package dev.langchain4j.model.input.structured
- *Annotation* pour un type (classe)
- Simplifie la création de prompts complexes en les définissant dans une classe à part, en fonction des champs de la classe
- Attributs :
  - `String[]` value : template de prompt défini en une ou plusieurs lignes
  - `String` delimiter : si template défini en plusieurs lignes, elles seront jointes avec ce délimiteur (\n par défaut)

Exemple 1

```
@StructuredPrompt("Créer une recette de {{plat}} qui peut
être préparé en utilisant seulement {{ingredients}}")
public record PromptRecette(String plat,
                            List<String> ingredients) { }

// Méthode main qui utilise PromptRecette :
PromptRecette prompt = new PromptRecette("salade",
    Arrays.asList("concombre", "tomate", "feta",
                  "oignon", "olives"));
Prompt prompt =
    StructuredPromptProcessor.toPrompt(prompt);
String reponse = modele.chat(prompt.text());
System.out.println(reponse);
```


Exemple 2 (plusieurs lignes)

```
@StructuredPrompt({"Créer une recette de {{plat}} qui peut
être préparé en utilisant seulement {{ingredients}}.",
   "Structure ta réponse de cette façon :",
   "Nom de la recette : ...",
   "Description : ...",
   "Temps de preparation : ...",
   "Ingredients :",
   "- ...",
   "- ...",
   "Instructions:",
   "- ...",
   "- ..."
}) public record PromptRecette(..., ...) { }
```

Tout le reste est identique à l'exemple 1

Interface `ImageModel`

- Package dev.langchain4j.model.image
- Pour générer une image à partir d'un texte
- Une méthode abstraite
  `Response<Image> generate(String prompt)`
- Méthodes par défaut
  `Response<Image> generate(String prompt, int n)` : génère n images
  `Response<Image> edit(Image image, String prompt)` : modifie l'image selon le prompt ; en 2ème paramètre on peut ajouter un masque (type `Image`) qui délimite la zone à modifier dans l'image

Exemple avec Gemini (1/2)

```
ChatModel model = GoogleGenAiChatModel.builder()
    .modelName("gemini-2.5-flash-image")
    .apiKey(llmKey).build();
String descriptionImage = "...";
 reponse =
    model.chat(UserMessage.from(descriptionImage));
AiMessage aiMessage = reponse.aiMessage();
List<Image> images =
    GeneratedImageHelper.getGeneratedImages(aiMessage);
if (GeneratedImageHelper.hasGeneratedImages(aiMessage)) {
    System.out.println(images.size() + " images générées");
    System.out.println("Texte de la réponse : "
                       + aiMessage.text());
```

Réservé version payante API Gemini

Exemple avec Gemini (2/2)

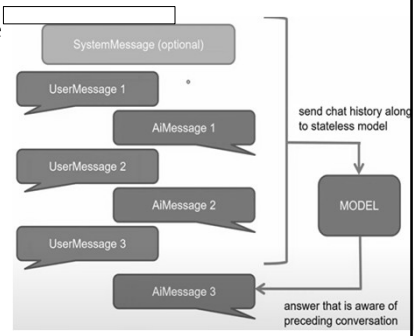
```
    int n = 1;
    for (Image image : images) {
        String base64Data = image.base64Data();
        String mimeType = image.mimeType();
        // On peut sauvegarder afficher, modifier image.
        // Pour la sauvegarder :
        byte[] imageBytes =
            Base64.getDecoder().decode(base64Data);
        Files.write(Paths.get("image" + n++ +".png"),
                    imageBytes);
    }
} else {
    System.out.println(aiMessage.text());
}
```


Exemple avec OpenAI

```
import dev.langchain4j.data.image.Image;
import dev.langchain4j.model.image.ImageModel;
import dev.langchain4j.model.openai.OpenAiImageModel;
import dev.langchain4j.model.output.Response;
import static
dev.langchain4j.model.openai.OpenAiImageModelName.DALL_E_3;
...
    ImageModel model = OpenAiImageModel.builder()
          .apiKey(ApiKeys.OPENAI_API_KEY)
          .modelName(DALL_E_3)
          .build();
    Response<Image> response = model.generate("...");
    System.out.println(response.content().url());
    }
}
```

Etat de la conversation

- Les APIs des LMs ne gardent pas l'état d'une conversation
- L'application chat doit donc garder cet état en envoyant le début de la conversation avec chaque requête



61

Etat et historique conversation

- L'API est sans état mais un état de la conversation peut être conservé par l'agent IA (client de l'API)
- L'enregistrement se fait à part, explicitement, dans une instance de type `ChatMemory`
- On peut vouloir conserver des informations différentes des messages exacts échangés Pour quelles raisons ?
- Par exemple, ne pas conserver des messages peu importants, ou ne conserver qu'un résumé de ce qu'il s'est passé pendant la conversation

62

Limiter la mémoire

- Pour des raisons de coûts ou pour simplifier la tâche du LM (enlever les informations inutiles), il peut être nécessaire de transformer la mémoire
- Par exemple on peut demander à un LM de résumer les messages les plus anciens (souvent les moins importants pour les dernières questions posées) lorsque la mémoire dépasse une certaine taille (en messages ou en tokens)

63

Interface `ChatMemory`

- Package dev.langchain4j.memory
- Représente l'état de la conversation durant un chat ; garde les derniers messages échangés avec le LLM
- Cet état peut être rendu persistant avec `ChatMemoryStore`

64

Méthodes de `ChatMemory`

- `void add(ChatMessage message)` : ajouter un message
- `List<ChatMessage> messages()` : liste des messages
- `void clear()` : supprime tous les messages
- `Object id()` : id de la chatMemory ; peut servir à distinguer 2 conversations qui se déroulent en parallèles

65

Implémentations `ChatMemory`

- Dans package dev.langchain4j.memory.chat :
  - `MessageWindowChatMemory` : ne retient que les N derniers messages
  - `TokenWindowChatMemory` : ne retient que les N derniers tokens ; nécessite un Tokenizer pour évaluer le nombre de tokens
- Par défaut, un seul `SystemMessage` ; si un nouveau message système est ajouté, l'ancien est supprimé
- Si le quota est atteint, le plus ancien item (message ou token) est supprimé de la mémoire
- La mémoire est conservée dans `ChatMemoryStore` ; par défaut `SingleSlotChatMemoryStore` est utilisé (conserve en mémoire centrale)

66

Création d'une mémoire

- Les 2 classes fournies utilisent le pattern « builder »
- Le builder permet
  - de donner un id (de type `Object`) pour distinguer plusieurs mémoires
  - d'indiquer le type de `ChatMemoryStore` à utiliser pour conserver les messages
  - de fixer le nombre maximum de messages ou de tokens à conserver
- Le plus souvent on crée une instance en fixant le nombre maximum de messages ou de tokens avec les méthodes `static withMaxMessages` ou `withMaxTokens`, suivant la classe

67

Exemple

```
ChatMemory memoire =
    MessageWindowChatMemory.withMaxMessages(10);
String question = "Films dirigés par Spielberg ?";
// Question ajoutée à la mémoire
memoire.add(UserMessage.from(question));
reponse = modele.chat(memoire.messages());
// AiMessage extrait de la réponse et ajouté à la mémoire
memoire.add(reponse.aiMessage());
// Nouvelle question
question = "En quelle année est-il né ?";
memoire.add(UserMessage.from(question));
reponse = modele.chat(memoire.messages());
System.out.println(reponse.aiMessage().text());
```

On verra qu'avec les Ai Services, la mémoire peut être gérée automatiquement

68

Mémoire persistante / à long terme

- Possible de créer une mémoire qui persiste entre plusieurs sessions avec une classe qui implémente `ChatMemoryStore` :

```
ChatMemory chatMemory =
  MessageWindowChatMemory.builder()
    .id("12345")
    .maxMessages(10)
    .chatMemoryStore(
       new PersistentChatMemoryStore())
    .build();
```



69

Interface `ChatMemoryStore`

- Package dev.langchain4j.store.memory.chat
- Implémentée par plusieurs classes dont `InMemoryChatMemoryStore`, `OracleChatMemoryStore`, `CassandraChatMemoryStore`
- Possible de récupérer des messages, de les supprimer ou de les modifier (utilise un id associé à chaque mémoire)
- Il est simple de créer une mémoire personnalisée en implémentant l'interface `ChatMemoryStore` ; par exemple pour enregistrer la mémoire dans une base de données relationnelle et ainsi garder l'historique en mémoire au-delà d'une seule session

70

Méthodes de `ChatMemoryStore`

- `void updateMessages(Object memoryId, List<ChatMessage> messages)` : met tous les messages de la liste dans la mémoire qui a cet id
- `List<ChatMessage> getMessages(Object memoryId)` : retrouve les messages pour la mémoire qui a cet id
- `void deleteMessages(Object memoryId)` : supprime tous les messages de la mémoire qui a cet id

71

Logging pour la sécurité

- Confier du travail à un logiciel qui peut halluciner n'est pas sans risques !
- Il est donc indispensable de garder une trace des échanges avec les parties des programmes qui utilisent des LMs, pour
  - Lire et évaluer leurs réponses
  - Comprendre pourquoi une erreur s'est produite
  - Améliorer leur comportement

72

Utilisation du logging

- Méthodes des builders des modèles pour indiquer si on veut activer le logging sur les interactions avec le modèle
- Dépend du modèle utilisé ; pour `OpenAIChatModel` et `GoogleGenAiChatModel`
  - `logRequests(boolean)`
  - `logResponse(boolean)`
- En plus pour `GoogleGenAiChatModel`
  - `logRequestsAndResponses(boolean)`

Exemple

```
ChatModel model = GoogleGenAiChatModel.builder()
    .apiKey(openAiKey)
    .modelName(...)
    .logRequests(true)
    .logResponses(true)
    .build();
```



Configuration logging

- Pour que le logging fonctionne, il faut le configurer (sinon un avertissement s'affiche : « SLF4J(W): No SLF4J providers were found. SLF4J(W): Defaulting to no-operation (NOP) logger implementation)
- LangChain4j utilise SLF4J (Simple Logging Facade for Java ; https://www.slf4j.org/), façade pour de nombreux frameworks de logging
- Il faut choisir une des APIs de logging supportées par SLF4J ; par exemple, Logback (le plus utilisé), celui du JDK (java.util.logging, JUL) ou Log4j
- Pour configurer le logging, il faut configurer le système de logging sous-jacent

Dépendance Maven pour logging

- Il suffit d'ajouter une dépendance vers l'API utilisée
- Par exemple, pour Logback

```
<dependency>
    <groupId>ch.qos.logback</groupId>
    <artifactId>logback-classic</artifactId>
    <version>****</version>
</dependency>
```

- Ou bien pour java.util.logging :

```
<dependency>
  <groupId>org.slf4j</groupId>
  <artifactId>slf4j-jdk14</artifactId>
  <version>****</version>
</dependency>
```


Configuration par code, Logback

```
import ch.qos.logback.classic.Logger;

private ListAppender<ILoggingEvent> appender;
...
private static void configureLogger() {
  Logger logger = (Logger)
      LoggerFactory.getLogger("dev.langchain4j");
  logger.setLevel(Level.INFO);
  LoggerContext context = logger.getLoggerContext();
  this.appender = new ListAppender<>();
  appender.setContext(context);
  appender.start();
  logger.addAppender(appender);
}
```

Un `Appender` peut permettre de récupérer les logs dans une `String` ou une liste

Configuration par code, JUL

```
import java.util.logging.Logger;
...
private static void configureLogger() {
  // Configure le logger sous-jacent (java.util.logging)
  // pour les types de LangChain4j
  Logger packageLogger =
      Logger.getLogger("dev.langchain4j");
  packageLogger.setLevel(Level.FINE); // Ajuster niveau
  // Ajouter un handler pour la console pour faire
  // afficher les logs
  ConsoleHandler handler = new ConsoleHandler();
  handler.setLevel(Level.FINE);
  packageLogger.addHandler(handler);
}
```

Modèles d'embeddings

- Il faut choisir le modèle qui convient le mieux à ce que l'on veut :
  - A quoi vont servir les embeddings
  - Coûts
  - Rapidité
  - Efficacité

Types d'embeddings Langchain4j

- Modèles **qui s'exécutent à part** (distants ou locaux) et que l'on utilise avec une API ; par exemple `GoogleGenAiEmbeddingModel`, `OpenAiEmbeddingModel`, `CohereEmbbdingModel`, `HuggingFaceEmbeddingModel`
- Modèles **intégrés** (« in-process ») qui s'exécutent dans le même processus que le code qui les utilisent ; une dizaine dans le package dev.langchain4j.model.embedding.onnx, par exemple `AllMiniLmL6V2EmbeddingModel` ou `AllMiniLmL6V2QuantizedEmbeddingModel`

Dépendance pour modèle intégré

- Pour utiliser un modèle d'embeddings intégré, il faut ajouter une dépendance Maven
- Par exemple, pour le modèle `BgeSmallEnV15EmbeddingModel`, la dépendance langchain4j-embeddings-bge-small-en-v15

Choisir un modèle d'embedding

- La taille des vecteurs générés est fixe ; elle est donnée par le getter `dimension()` de `EmbeddingModel`
- Plus la dimension est grande, plus l'embedding peut porter du sens, mais plus les coûts en calcul et en mémoire sont importants
- Préciser le type de tâche pour laquelle on veut utiliser le modèle, modifie légèrement les vecteurs générés ; types pour `GoogleGenAiEmbeddingModel.TaskTypeEnum` : `SEMANTIC_SIMILARITY`, `CLASSIFICATION`, `CLUSTERING`, `RETRIEVAL_DOCUMENT` (RAG, phase 1), `QUESTION_ANSWERING` (RAG, phase 2), `FACT_VERIFICATION`



Taille des embeddings

- Exemples de tailles : 256 (si mémoire limitée), 384 (petits modèles, en particulier les modèles intégrés), 512, 768 (courant), 1024, 1536, 3072
- Certains modèles d'embedding (en particulier les modèles de Google) permettent de réduire la taille si on veut absolument réduire l'occupation mémoire ou/et accélérer les recherches sémantiques ; les vecteurs d'origine sont projetés sur un espace vectoriel plus petit ; on perd alors un peu d'information mais on dépense moins de ressources

Exemple modèle d'embedding

```
EmbeddingModel modele = GoogleGenAiEmbeddingModel.builder()
  .apiKey(llmKey)
  .modelName("gemini-embedding-001")
  .taskType(
      GooglegenAiEmbeddingModel.TaskTypeEnum.SEMANTIC_SIMILARITY)
  .timeout(Duration.ofSeconds(2))
  .build();
String phrase1 = "Bonjour, comment allez-vous ?";
String phrase2 = "Salut, quoi de neuf ?";
Response<Embedding> reponse1 = modele.embed(phrase1);
Response<Embedding> reponse2 = modele.embed(phrase2);
Embedding emb1 = reponse1.content();
Embedding emb2 = reponse2.content();
// Calcul de similarité cosinus entre les 2 embeddings
double similarite = CosineSimilarity.between(emb1, emb2);
System.out.println("Similarité cosinus : " + similarite);
```

Exemple modèle intégré

```
// Même code que l'exemple précédent, sauf pour la
// création du modèle d'embedding ; nécessite dépendance
// Maven langchain4j-embeddings-all-minilm-l6-v2)
// Pour le modèle BgeSmallEnV15EmbeddingModel, c'est
// la dépendance Maven
// langchain4j-embeddings-bge-small-en-v15
EmbeddingModel modele =
    new AllMiniLmL6V2EmbeddingModel();
```


85

AiServices

86

Chaînes et services IA

- Pour créer des flux automatisés et structurés autour des LMs
- Chaînes : inspirées de LangChain ; permettent de combiner plusieurs étapes d'utilisation des LMs ; pas étudiées dans ce support
- Services IA (construits avec la classe `AiServices`) : autre solution que les chaînes, mieux adaptée à Java que les chaînes

87

Service IA

- Le développeur écrit une interface Java dont les méthodes correspondent aux interactions avec le LM
- Il configure ces interactions et **LangChain4j se charge d'implémenter l'interface** en générant le code pour envoyer les requêtes au LM et pour récupérer les réponses
- LangChain4j tient compte des types des paramètres, du type retour, des annotations des méthodes de l'interface, et de la configuration fournie à `AiServices` pour écrire l'implémentation des méthodes

Comment est-ce possible ?

88

Exemple interface et création instance de l'interface

```
public interface Assistant {
  String chat(String prompt);
}
...
Assistant assistant =
  AiServices.builder(Assistant.class)... // config
```

On peut choisir le nom de l'interface et de ses méthodes

- `assistant.chat(question)` enverra la question au LM et la valeur retournée par `chat` représentera la réponse du LM

89

Configuration assistant

- L'assistant peut être configuré en ajoutant des annotations dans l'interface ou avec le builder de `AiServices`, par exemple pour utiliser une mémoire ou faire du RAG
- Un exemple minimal de configuration avec le builder :

```
Assistant assistant =
  AiServices.builder(Assistant.class)
            .chatModel(modele)
            .build();
```


90

Supporté par AiServices

- Presque tout ce qui est fait avec l'API de bas niveau de LangChain4j peut être fait avec les AI services :
  - Mémoire pour la conversation (peut être gérée automatiquement)
  - RAG (`RetrievalAugmentor` et `ContentRetriever`)
  - Outils (Tools)
  - Streaming
  - Auto-modération
  - Guardrails
  - ...



Classe AiServices<T>

- Package dev.langchain4j.service
- Cette classe crée un service IA en implémentant une interface définie par le développeur (création avec méthode `create` pour les cas simples, ou avec le modèle de conception « builder »)
- `T` est l'interface pour laquelle `AiServices` va fournir une implémentation
- La classe contient de très nombreuses méthodes de configuration

Méthodes de AiServices<T> (1/6)

- `public static <T> T create(Class<T> aiService, ChatModel chatModel)` :
  crée simplement un service IA pour le modèle indiqué ; variante avec `StreamingChatModel` pour le 2ème paramètre
- `public static <T> AiServices<T> builder(Class<T> aiService)` : pour création plus complexe avec le modèle de conception « builder »
- `public abstract T build()` : construit et retourne le service IA (après configuration par modèle « builder »

Méthodes de AiServices<T> (2/6)

- `public AiServices<T> chatModel(ChatModel model)` : indique le modèle qui sera utilisé par le service IA
- `public AiServices<T> streamingChatModel( StreamingChatModel model)` : variante pour streaming
- `public AiServices<T> chatMemory(ChatMemory chatMemory)` : indique la mémoire qui sera utilisée par le service IA (pas de mémoire par défaut) ; si on veut une mémoire différente pour chaque user, utiliser la méthode `chatMemoryProvider` qui prend en paramètre un `ChatMemoryProvider` (voir exemple à suivre)

Memory pour chaque utilisateur

```
interface Assistant {
  String chat(@MemoryId int memoryId,
              @UserMessage String userMessage);
}
Assistant assistant = AiServices.builder(Assistant.class)
   .chatModel(OpenAiChatModel.withApiKey(...))
   .chatMemoryProvider(memoryId ->
         MessageWindowChatMemory.withMaxMessages(10))
   .build();
System.out.println(
    assistant.chat(1, "Hello, my name is Jean"));
    // Hi Jean! How can I assist you today?
System.out.println(
    assistant.chat(2, "Hello, my name is Francine"));
    // Hello Francine! How can I assist you today?
System.out.println(assistant.chat(1, "What is my name?"));
    // Your name is Jean.
System.out.println(assistant.chat(2, "What is my name?"));
    // Your name is Francine.
```


@MemoryId

- Annotation du package dev.langchain4j.service, qui peut se mettre sur un des paramètres d'une méthode d'une interface utilisée par AI services
- La valeur du paramètre annoté sera utilisée pour identifier la mémoire qui est associée à une conversation/un utilisateur
- Le paramètre peut être de n'importe quel type
- Le `ChatMemoryProvider` doit être configuré pour les AI service (voir code précédent)

Méthodes de AiServices<T> (3/6)

- `public AiServices<T> tools(List<Object> objectsWithTools)` : configure les outils que le LM pourra utiliser ; une mémoire d'au moins 3 messages est requise
- `public AiServices<T> contentRetriever(ContentRetriever contentRetriever)` : configure un retriever qui sera appelé pour chaque exécution de méthode du service IA pour retrouver le contenu associé à un message utilisateur depuis une source de données (par exemple un magasin d'embeddings dans le cas d'un `EmbeddingStoreContentRetriever`)

Méthodes de AiServices<T> (4/6)

- `public AiServices<T> retrievalAugmentor(RetrievalAugmentor retrievalAugmentor)` : configure un `RetrievalAugmentor` qui sera appelé pour chaque exécution de méthode du service IA ; un `RetrievalAugmentor` ajoute un `UserMessage` avec un contenu retrouvé par un retriever ; on peut utiliser le `DefaultRetrievalAugmentor` fourni par LangChain4j ou en implémenter un autre

Méthodes de AiServices<T> (5/6)

- `public AiServices<T> chatRequestTransformer(UnaryOperator<ChatRequest> transformer)` : configure un transformeur qui sera appliqué à la `ChatRequest` avant d'être envoyée au LM (pour ajouter ou modifier des messages) ; peut aussi prendre en paramètre un transformeur de type `BiFunction<ChatRequest, Object, ChatRequest>` (`Object` représente un memory ID qui peut être utilisé pour retrouver des informations de la mémoire)
- Autres méthodes liées à la modération, aux guardrails (vérifient ce qui est envoyé au LM et ce qui est)

Méthodes de AiServices<T> (6/6)

- Autres méthodes liées
  - à la modération
  - aux guardrails : vérifient ce qui est envoyé au LM et ce qui est retourné par le LM
  - à l'ajout d'écouteurs qui sont déclenchés par les événements qui surviennent pendant le traitement des requêtes envoyées au LM
  - aux outils : exécution séquentielle ou en // des outils (séquentielle par défaut) ; traitement des erreurs ; fournisseur d'outils (voir MCP)
  - au RAG (est-ce que ce qui a augmenté les messages doivent être gardé en mémoire ; oui par défaut)

Exemple simple

```
public interface Aide {
    String demander(String prompt);
}
```

```
Aide aide = AiServices.builder(Aide.class)
                      .chatModel(modele)
                      .build();
String reponse = aide.demander("Hello, world");
System.out.println(reponse);
```


Exemple simple avec mémoire

```
ChatModel modele = GoogleGenAiChatModel.builder()
                   .apiKey(cle)
                   .modelName("gemini-3.5-flash")
                   .build();
ChatMemory memoire =
    MessageWindowChatMemory.withMaxMessages(10);
Assistant assistant = AiServices.builder(Assistant.class)
                      .chatModel(modele)
                      .chatMemory(memoire)
                      .build();
String rep1 = assistant.chat("Hello! je m'appelle Julie");
System.out.println(rep1); // Hello Julie !
String rep2 = assistant.chat("Quel est mon nom?");
System.out.println(rep2); // Votre nom est Julie.
```

Comment ça marche ?

- A l'exécution, `AiServices` crée un objet proxy qui implémente l'interface (en utilisant la réflexivité)
- Le proxy s'occupe, entre autres,
  - d'envoyer les requêtes au LM dans un format compatible avec le LM (le plus souvent JSON)
  - de recevoir les réponses du LM (le plus souvent JSON)
  - de gérer les erreurs et exception
  - de convertir les paramètres et les valeur retour des méthodes de l'interface ; par exemple pour transformer une `String` en `UserMessage`, en entrée, et un `AiMessage` en `String`, en sortie

Méthodes de l'interface

- Elles définissent les interactions avec le service IA
- Elles peuvent être annotées par `@UserMessage` ou `@SystemMessage` pour définir un template pour un `UserMessage` ou un `SystemMessage` (avec des variables pour les éventuels paramètres de la méthode) qui seront envoyés au LM dans chaque requête ; `@UserMessage` peut aussi être mis sur un paramètre
- Contraintes sur les types des paramètres et le type retour des méthodes car elles correspondent à une requête qui est envoyée au LM

Paramètres des méthodes

- De type `String` (ou `Content` pour requêtes multimodales) ou d'un type qui peut être transformé par LangChain4j en un ou plusieurs prompts
- Un paramètre non annoté correspond à un ou plusieurs `UserMessage` envoyé au LM (on peut aussi l'annoter avec `@UserMessage`)
- Si le texte du `UserMessage` (ou `SystemMessage`) est un template, les valeurs des variables du template sont alors définies par les paramètres de la méthode de type `String` annotés par `@V`

Cas simple de méthode

- `String chat(String userMessage)`
- Un seul paramètre pour un `UserMessage`



Exemples de méthodes (1/3)

- `String chat(@UserMessage String userMessage)`
  puisqu'il n'y a qu'un seul paramètre, identique à
  `String chat(String userMessage)`
- `@SystemMessage("Donner le nom de la capitale du pays indiqué, et rien d'autre")`
  `String chat(String pays);`
  `@SystemMessage` peut aussi s'appliquer à un paramètre s'il peut changer dynamiquement
- `@UserMessage("Valeur de {{info}} sur {{pays}} ?")`
  `String chat(@V("info") String info, @V("pays") String pays);`
  `@UserMessage` contient un template. Les paramètres du template sont les paramètres de la méthode, annotés par `@V`

Exemples de méthodes (2/3)

- `@UserMessage("Valeur de {{info}} sur {{pays}} ?")`
  `String chat(@V("info") String info, @V("pays") String pays);`
  `@UserMessage` contient un template. Les paramètres du template sont les paramètres de la méthode, annotés par `@V`
  On pourra l'utiliser comme ceci :
  `assistant.chat(population, France);`
- Multimodalité :

```
String chat(@UserMessage String userMessage,
            @UserMessage ImageContent image,
            @UserMessage AudioContent audio);
```

Exemples de méthodes (3/3)

```
@SystemMessage("You are a helpful assistant that always
replies in French.")
String chat(@MemoryId String sessionId,
            @UserMessage String userMessage
);
```

```
@SystemMessage("Donner le nom de la capitale du pays
indiqué, et rien d'autre")
@UserMessage("{{pays}}") // template...
String chat(@V("pays") String pays);
```

```
@SystemMessage("Pour le pays donné, {{question}}")
@UserMessage("{{pays}}")
String chat(@V("question") String question,
            @V("pays") String pays);
```


Exemple complet

```
interface Traducteur {
  @SystemMessage("""
    Tu es un traducteur professionnel de français en
    {{langue}}""")
  @UserMessage("Traduis le texte suivant: {{texte}}")
  String traduire(@V("texte") String texte,
                  @V("langue") String langue);
 }
```

```
Traducteur traducteur = AiServices.builder(Traducteur.class)
                        .chatModel(modele).build();
```

```
String italien = traducteur.traduire("Hello", "italien");
```


Types retour méthodes interface

- `String`, `AiMessage`, `Result` ; `Result` permet d'avoir en plus `TokenUsage`, métadonnées ou `FinishReason`
- Nombreux autres types possibles :
  - `List<String>` ou `Set<String>`
  - `Map<K,V>`
  - Une énumération ou un `boolean` (par exemple si on veut utiliser le LM pour une classification)
  - Un type primitif, une classe qui enveloppe un type primitif, `Date`, `LocalDateTime`, `BigDecimal`, ...
  - Ou même une classe quelconque (POJO) ; voir « Extraction de données » plus loin

Type `Result<T>`

- Package dev.langchain4j.service
- `T` est le type supporté par les services IA (`AIServices`) en type retour : `String`, `AiMessage`, `Enum`, ...
- Enveloppe le type `T` ; permet d'avoir des informations supplémentaires sur le traitement de la requête : `FinishReason`, `TokenUsage`, les sources (`List<Content>`) si RAG, les outils utilisés (`List<ToolExecution>`)

Méthodes de `Result<T>`

- `T content()` : contenu de la réponse
- `ChatResponse finalResponse()` : réponse du `ChatModel`
- `FinishReason finishReason()` (extrait de `ChatResponse`)
- `TokenUsage tokenUsage()` : coût total en tokens
- `List<Content> sources`) : les sources si RAG
- `List<ChatResponse> intermediateResponses()` : toutes les réponses intermédiaires du LM
- `List<ToolExecution> toolExecutions()` : outils utilisés

Exemple avec `Result<T>`

```
public interface AgentMeteo {
  Result<String> chat(String requete);
}
```

```
AgentMeteo agentMeteo =
    AiServices.builder(AgentMeteo.class)
              .chatModel(model)
              .tools(new MeteoTool())  // Ajout outil
              .build();
Result<String> resultat =
    agentMeteo.chat("Quel temps fait-il à Nice ?");
System.out.println(resultat.content()); // Réponse LM
TokenUsage tokenUsage = resultat.tokenUsage();
List<ToolExecution> executions = resultat.toolExecutions;
for (ToolExecution toolExecution : executions) {
  System.out.println(toolExecution.toString());
}
```

Sorties structurées
Extraction de données

115

Type structuré en réponse

- Pour plus de sécurité et de facilité de traitement par l'agent IA, il est souvent intéressant de récupérer les réponses du LM sous la forme d'un objet Java structuré plutôt que sous la forme d'une `String`
- Particulièrement utile pour garantir que les réponses suivent un format attendu, comme un objet JSON, une liste ou une classe spécifique
- La réponse sera ainsi plus facilement utilisée par un logiciel

116

Types standard supportés

- Types retour standards supportés par LangChain4j pour les méthodes des interfaces utilisées par `AIServices`
- L'exemple suivant montre comment récupérer une date (`LocalDate`) ou un temps (`LocalTime`) d'un texte

117

`ExtracteurTemps` (1/2)

```
public interface ExtracteurTemps {
  @UserMessage("Extrait la date de {{it}}")
  LocalDate extraireDate(String texte);

  @UserMessage("Extrait le temps de {{it}}")
  LocalTime extraireTemps(String texte);

  @UserMessage("Extrait la date et le temps de {{it}}")
  LocalDateTime extraireDateEtTemps(String texte);
}
```


118

`ExtracteurTemps` (2/2)

```
ExtracteurTemps extracteur =
    AiServices.builder(ExtracteurTemps.class)
              .chatModel(model)
              .build();
String texte = """
  La tranquillité régnait dans la soirée de 1995, à un
  quart d'heure de minuit, le jour après Noël.""");

LocalDate date = extracteur.extraireDate(texte);
LocalTime temps = extracteur.extraireHeure(texte);
// Que sera-t-il affiché ?
System.out.println(date + "; " + temps);
```


119

Pour les classes Java

- L'extraction fonctionne aussi sous la forme de classes ou de records Java, créés dans l'application
- Voici, par exemple, un texte :
  Christophe Colomb, né en 1451 sur le territoire de la république de Gênes et mort le 20 mai 1506 à Valladolid, est un navigateur génois au service des Rois catholiques, Isabelle de Castille et Ferdinand d'Aragon.
- Voyons comment récupérer dans un record Java, le nom, les dates et lieux de naissance du personnage dont on parle

120

Étapes

1. Créer le modèle **en demandant une réponse au format JSON**
2. Écrire la structure de l'information à récupérer sous la forme d'une classe ou d'un record Java, **en utilisant des noms significatifs (ou `@Description`)**
3. Écrire l'interface de l'extracteur d'information, utilisé par `AiServices`
4. Lancer l'exécution de l'extracteur

Créer le modèle en demandant une réponse au format JSON

Exemple, étape 1

```
ChatModel modele =
 GoogleGenAiChatModel
    .builder()
    .apiKey(llmKey)
    .modelName("gemini-3.5-flash")
    .responseFormat(ResponseFormat.JSON)
    .build();
```


Structure de l'information à récupérer sous la forme d'un record, en utilisant des noms significatifs

Exemple, étape 2

```
public record Personne(String nom,
                       int anNaissance,
                       String lieuNaissance,
                       LocalDate dateMort) { }
```


Écrire l'interface de l'extracteur d'information

Exemple, étape 3

```
public interface ExtracteurInfosPersonne {
  @UserMessage("""
    Extrait les informations sur la personne concernée
    par le texte ci-dessous :
    ---
    {{it}}
    ---
    """)
    Personne extraireInfosPersonne(String texte);
}
```

Que faut-il modifier si on veut aussi avoir l'usage des tokens ?

Lancer l'exécution de l'extracteur

Exemple, étape 4

```
ExtracteurInfosPersonne extracteur =
    AiServices.builder(ExtracteurInfosPersonne.class)
              .chatModel(modele).build();

Personne personne = extracteur.extraireInfosPersonne (
    """
    Christophe Colomb, né en 1451 sur le territoire de la république de
    Gênes et mort le 20 mai 1506 à Valladolid, est un navigateur génois
    au service des Rois catholiques, Isabelle de Castille et Ferdinand
    d'Aragon.
    """);
System.out.println(personne.nom());
System.out.println(personne.anNaissance());
```


Description pour extraction

- Si le type retour est un type structuré Java et si le contenu d'un attribut n'est pas clair d'après son nom, on peut une annotation `@Description` pour aider le LM à faire son travail
- Par exemple

```
public record Personne(
    String nom,
    @Description("Année de naissance") int anNaiss,
    String lieuNaissance) {
}
```

Réponse structurée avec métadonnées

- Pour avoir en plus les informations sur les tokens, les sources (si RAG), les exécutions d'outils,..., il suffit d'envelopper le type retour Java avec `Result` : `Result<Personne> extraireInfosPersonne(String texte);`

Outils

Rappel

- Les LMs peuvent utiliser des outils pour agir sur l'extérieur ou étendre leurs capacités, en collaboration avec un agent IA
- Le processus pour utiliser un outil est complexe avec 5 étapes, dont 2 requêtes (et donc 2 réponses)

  Pourquoi 2 requêtes ?

Apport de LangChain4j

- LangChain4j simplifie le processus :
  - La méthode Java de l'outil est annotée par `@Tool` qui décrit la fonction en langage naturel
  - Tout le reste est automatisé par LangChain4j

Exemple - la question

- Question : quelle est la racine carrée de la somme des nombres des lettres dans les mots « bonjour » et « Monsieur » ? (la réponse est $\sqrt{15}$)

Exemple

```
Assistant assistant = AiServices.builder(Assistant.class)
    .chatModel(modele)
    .tools(new Calculator())
    .chatMemory(MessageWindowChatMemory.withMaxMessages(10))
    .build();
String question = "Quelle est la racine carrée des nombres de lettres dans les mots 'bonjour' et 'Monsieur' ? ;
String answer= assistant.chat(question);
```

```
interface Assistant {
  String chat(String userMessage);
}
```

Exemple - code de 3 outils

```
public class Calculator {
  @Tool("Calcule la longueur d'une chaîne de caractères")
  public int stringLength(String s) {
    return s.length();
  }
  @Tool("Calcule la somme de deux entiers")
  public int add(int a, int b) {
    return a + b;
  }
  @Tool("Calcule la racine carrée d'un entier")
  public double sqrt(int x) {
    return Math.sqrt(x);
  }
}
```


133

@P (1/2)

- Package dev.langchain4j.agent.tool
- Décrit un paramètre d'un outil
- Recommandé d'utiliser cette annotation
- L'attribut `value` ou `description` de `@P` donne une description en langage naturel du paramètre, pour aider le LM à choisir les outils qu'il va utiliser et à les utiliser correctement

134

@P (2/2)

- `@P` peut avoir un attribut `required` pour indiquer si le paramètre est requis (requis par défaut) ; s'il est requis, le LM devra fournir ce paramètre pour utiliser l'outil sinon, l'attribut `defaultValue` permet de donner une valeur par défaut
- Le nom Java du paramètre est perdu dans la compilation sans l'option « –parameters » de javac ; les arguments ont alors les noms argo, arg1, etc. ; l'attribut `name` permet de donner un nom plus significatif

135

Exemple @P

```
class OutilMétéo {
    @Tool("Renvoie les prévisions météo pour une ville")
    pubic String previsions(
        @P(value="Ville dont on veut les prévisions météo",
            name="ville") required=true)
        String ville,
        @P("Unité de temperature, Celcius ou Fahrenheit",
            name="unit"
           required="false", defaultValue="Celcius")
        TemperatureUnit unit) {
        ...
    }
}
```



136

Exemple avec devises (1/2)

```
@Tools("Convertit un montant monétaire d'une devise
devise1 vers une autre devise devise2 ")
public double convert(double montant, @P("devise1")
String devise1, @P("devise2") String devise2) {
  return montant * taux.get(devise1) / taux.get(devise2);
}

@Tool("Obtient le nom d'une devise à partir de son
abréviation")
public String getCurrency(String abreviation) {
  return devises.get(abreviation);
}
```


137

Exemple avec devises (2/2)

```
String question = """
   Sur le site Vente.com de plusieurs pays, un ordinateur
   coûte 718,25 GBP, 83900 INR, 749,99 USD,
   et 177980 JPY. Quelle est la meilleure offre ?
   Dans le résultat, écris le nom de la devise et le
   montant dans cette devise.
""";
String reponse = assistant.chat(question);
```


138

@Tool

- Package dev.langchain4j.agent.tool
- Définit la méthode comme un outil et décrit ce qu'elle fait
- Principalement 2 paramètres optionnels :
  - `name` est le nom de l'outil ; par défaut le nom de la méthode de l'outil
  - `value` donne une description en langage naturel de l'outil pour aider le LM à choisir les outils qu'il va utiliser ; par défaut déduit du nom de la méthode

@Description

- Package dev.langchain4j.model.output.structured
- Peut annoter une classe, ou un champ d'une classe
- Par exemple, pour décrire une classe (et ses champs) qui est le type d'un paramètre d'une méthode d'outils, ce qui peut aider le LM à choisir s'il veut utiliser l'outil, et comment
- On a vu que cette annotation peut aussi servir pour décrire des champs de structures de données dans l'extraction des données

Exemple @Description

```
@Description("Requête à exécuter")
class Requete {

  @Description("Les champs à sélectionner")
  private List<String> select;

  @Description("Conditions de filtrage")
  private List<Condition> where;
}

@Tool
Resultat executeQuery(Requete requete) {
  ...
}
```


Types paramètres des outils

- Types primitifs
- `String`, enveloppes des types primitifs
- Enumérations
- Classe quelconque (doit permettre la conversion en JSON)
- `List<T>`, `Set<T>` (avec `T` d'un des types ci-dessus)
- `Map<K,V>` (on doit spécifier les types `K` et `V` dans la description du paramètre avec `@P`)
- L'idée est que le type doit pouvoir être converti en JSON

Type retour des outils

- Si le type retour est `String`, la valeur retournée est envoyée au LM
- Si le type retour est `void`, la chaîne « Success » est envoyée au LM
- Sinon, la valeur retournée est sérialisée en JSON et envoyée au LM

Gestion des erreurs dans outils

- Si une méthode annotée avec `@Tool` lève une exception, le message de l'exception (`e.getMessage()`) sera envoyé au LM à la suite de l'exécution de l'outil
- Le LM pourra ainsi corriger son erreur et réessayer, s'il le juge nécessaire

@ToolMemoryId

- Si un paramètre d'une méthode d'un outil est annoté avec cette annotation, le paramètre aura la valeur du memory id (la mémoire de la conversation ; voir AI services)
- Permettra de distinguer les différentes conversations

Définir outil par programmation

- Pour les cas particuliers où les outils sont chargés pendant l'exécution depuis des sources externes, on peut les définir en utilisant `ToolSpecification`
- Il faut alors fournir une implémentation de `ToolExecutor`

Exemple - ToolSpecification

```
ToolSpecification toolSpecification =
    ToolSpecification.builder()
    .name("get_booking_details")
    .description("Returns booking details")
    .parameters(JsonObjectSchema.builder()
        .properties(Map.of(
            "bookingNumber",
            JsonStringSchema.builder()
            .description("Booking number in B-12345 format")
            .build()
         ))
         .build())
    .build();
```


Exemple - ToolExecutor

```
ToolExecutor toolExecutor =
    (toolExecutionRequest, memoryId) -> {
        Map<String, Object> arguments =
            fromJson(toolExecutionRequest.arguments());
        String bookingNumber =
            arguments.get("bookingNumber").toString();
        Booking booking = getBooking(bookingNumber);
        return booking.toString();
};
```


Exemple - création service IA

```
Assistant assistant = AiServices.builder(Assistant.class)
    .chatModel(chatModel)
    .tools(Map.of(toolSpecification, toolExecutor))
    .build();
```


Définir outils dynamiquement

- Des outils peuvent être ajoutés dynamiquement, selon le contenu de chaque requête
- Il faut pour cela utiliser un `ToolProvider`

Appel d'outil inexistant

- Pendant le traitement d'une requête le LM peut halluciner et demander l'utilisation d'un outil inexistant
- Par défaut une exception sera alors lancée
- On peut changer ce comportement par défaut en définissant une stratégie avec la méthode `hallucinatedToolNameStrategy` du builder de `AiServices`, au moment de la création du service IA

151

Service IA comme outil

- Un service IA peut être annoté avec `@Tool` et il peut alors être utilisé comme un outil

152

Exemple

```
public interface Assistant {
  @Tool("Répond à des questions qui concerne …")
  @SystemMessage(….)
  Result<String> chat(String question)
}
```


153

Erreur pendant exécution

- Erreur de code 400, INVALID_ARGUMENT "Please ensure that function call turn comes immediately after a user turn or after a function response turn."
- Cette erreur peut être due au fait que l'historique ne garde pas suffisamment de messages
- Il est donc bon de commencer par augmenter les messages conservés dans l'historique

154

Précautions

- L'utilisation d'outils peut générer un grand nombre de tokens (voir TP 2), ce qui peut occasionner un coût et de rendre le LM moins attentif à certaines informations présentes dans le contexte (« Lost in the middle »)
- Il faut donc sélectionner avec précaution les outils qu'on présente au LM
- LangChain4j offre la notion de stratégie de recherche d'outils qui permet de limiter les outils présentés au LM

155

Modération

156

Présentation

- Dans une application d'entreprise il est important de s'assurer que les contenus échangés avec les clients et partenaires de l'entreprise sont appropriés et sûrs
- L'entreprise ne doit pas injurier ses clients et elle ne doit pas répondre à des propos dangereux, injurieux, racistes ou sexistes

157

Comment modérer

- Il est simple de « modérer » automatiquement les conversations avec `AiServices` :
  1. Créer un `ModerationModel`
  2. A la création de l'assistant par `AiServices`, indiquer que l'on veut modérer les conversations avec ce `ModerationModel`
  3. Il suffit ensuite d'ajouter l'annotation `@Moderate` sur les méthodes de l'assistant qui seront modérées



158

Classe `Moderation`

- Package dev.langchain4j.model.moderation
- Représente le statut de la modération qui indique le texte qui a été modéré et s'il a été signalé comme problématique (flagged)

159

`@Moderate`

- Package dev.langchain4j.service
- Quand une méthode d'un assistant est annotée avec cette annotation, chaque appel de la méthode déclenche un appel en // au modèle de modération
- Avant la fin de la méthode, le résultat du modèle de modération est attendu
- Si le modèle de modération signale un problème, une `ModerationException` est lancée par la méthode ; la méthode `moderation()` de cette exception retourne une instance de `Moderation`

160

Interface `ModerationModel`

- Package dev.langchain4j.model.moderation
- Implémenté par `OpenAiModerationModel`, `MistralAiModerationModel`, `WatsonxModerationModel`, `DisabledModerationModel` (ce dernier uniquement pour tests ; toutes les méthodes lancent une exception)
- 5 méthodes `moderate` surchargées pour modérer un texte, ou un ou plusieurs messages de l'utilisateur

161

Classe `OpenAiModerationModel`

- Builder ou construction avec le nom du modèle qui est une des valeurs de l'énumération `OpenAiModerationModelName` :
  - `OMNI_MODERATION_LATEST`
  - `OMNI_MODERATION_2024_09_06`

162

Exemple (1/3)

```
interface Chat {
  @Moderate
  String chat(String text);
}
```


163

Exemple (2/3)

```
ModerationModel moderationModel =
    OpenAiModerationModel.builder()
        .apiKey(cleOpenAi)
        .modelName(OMNI_MODERATION_LATEST)
        .build();

ChatModel chatModel = ...;

Chat chat = AiServices.builder(Chat.class)
            .chatModel(chatModel)
            .moderationModel(moderationModel)
            .build();
```


164

Exemple (3/3)

```
try {
  chat.chat("Je vais être méchant avec tout le monde !!!");
} catch (ModerationException e) {
  System.err.println(e.getMessage());
  // Affiche "Je vais être méchant avec tout le monde !!!
  // violates content policy"
}
```


165

Guardrails

- Input
- Output

166

Utilité

- Valider ou filtrer les messages avant l'appel au LLM (entrée) ou après la génération (sortie)
- Indispensable pour la sécurité
- Souvent associé à la modération
- Disponible seulement pour les services IA (pas pour le plus bas niveau)
- Les vérifications peuvent être faites par du code Java ordinaire ou du code qui utilise un LM ; par exemple un LM spécialement entrainé à détecter les injections de prompt

167

Exemples

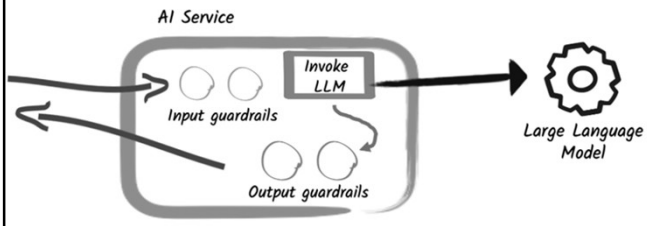
- Vérification des questions posées par l'utilisateur, en particulier contre les injections de prompt
- Vérification du format de sortie des réponses du LM et de la cohérence avec les règles et contraintes de l'entreprise ou gouvernementales
- Détection d'hallucinations

168

Implémentation LangChain4j

- Une classe Java par validation
- Ces classes sont chainées
- Il faut placer en premier dans la chaine les classes qui ont le plus de chance d'échouer
- 2 types de guardrail :
  - vérifie les entrées dans le LM ; implémente interface `InputGuardrail`
  - vérifie les sorties du LM ; implémente interface `OutputGuardrail`

169



Source https://docs.langchain4j.dev/tutorials/guardrails

170

Dépendance à ajouter

```
<dependency>
    <groupId>dev.langchain4j</groupId>
    <artifactId>langchain4j-guardrails</artifactId>
    <version>****</version>
</dependency>
```



171

Interface `InputGuardrail` (1/2)

- Package dev.langchain4j.guardrail
- 2 méthodes `default` (au moins une doit être implémentée) :
  - `InputGuardrailResult validate(UserMessage userMessage)` : pour les cas simples où on a seulement besoin du message utilisateur
  - `InputGuardrailResult validate(InputGuardrailRequest params)` : pour les cas où on a aussi besoin de la mémoire et d'autres valeurs pour valider

172

Interface `InputGuardrail` (2/2)

- Des méthodes `default` pour donner le résultat (type retour `InputGuardrailResult`) d'un guardrail :
  - `success()` : étape validée
  - `successWith(String)` : étape validée ; en paramètre le texte du succès
  - `failure(String)` : étape en échec ; en paramètre qui explique le problème
  - `failure(String, Throwable)` : paramètre en plus qui est la cause (au sens des `Exception`s Java) de l'échec
  - `fatal`, avec les mêmes paramètres que `failure`

173

`InputGuardrailRequest`

- Représente ce que l'on passe à la méthode `validate` de `InputGuardrail`

174

InputGuardrailException

- Classe du package dev.langchain4j.guardrail
- Lancée par LangChain4j quand une validation de guardrail échoue ; attrapée par le code écrit par le développeur

175

InputGuardrailResult (1/2)

- Classe du package dev.langchain4j.guardrail qui représente le résultat d'une validation
- La méthode `result()` retourne une instance de l'énumération `Guardrail.Result` qui a pour valeurs
  - `FATAL` : la validation a échoué et toutes les éventuelles évaluations à venir n'auront pas lieu
  - `FAILURE` : la validation a échoué ; les autres évaluations auront lieu
  - `SUCCESS` : la validation a réussi
  - `SUCCESS_WITH_RESULT` : la validation a réussi mais le message utilisateur a été modifié avant la prochaine validation

176

InputGuardrailResult (2/2)

- Des méthodes `static` permettent de créer une instance :
  - `success()`
  - `successWithMessage(String)`

177

Déclaration d'un InputGuardrail

- Par ordre de priorité des classes guardrails sont données :
  1. En paramètres de la méthode `inputGuardrailClasses` du builder de service IA, ou bien des instances des classes sont passées en paramètres la méthode `inputGuardrails` du builder
  2. En paramètres de l'annotation `@InputGuardrails` sur une des méthodes qui définit un service IA
  3. En paramètres de l'annotation `@InputGuardrails` sur l'interface qui définit un service IA

178

Processus de validation

- Le premier guardrail est exécuté
- Si son résultat est un succès, le LLM est appelé
- Le guardrail suivant est exécuté si le premier n'a pas le résultat « fatal »
- Un guardrail peut récrire le message utilisateur
- Si un guardrail a récrit le message utilisateur, le guardrail suivant reçoit ce message (à la place de l'ancien)

179

Exemple 1

```
public interface Assistant {
    String chat(String question);
    String doSomethingElse(String question);
}

var assistant = AiServices.builder(Assistant.class)
    .chatModel(chatModel)
    .inputGuardrailClasses(FirstInputGuardrail.class,
                           SecondInputGuardrail.class)
    .build();
```


180

Exemple 2

```
public interface Assistant {
    String chat(String question);
    String doSomethingElse(String question);
}

var assistant = AiServices.builder(Assistant.class)
    .chatModel(chatModel)
    .inputGuardrailClasses(new FirstInputGuardrail(),
                           new SecondInputGuardrail())
    .build();
```


181

Exemple 3

```
public interface Assistant {
    @InputGuardrails({ FirstInputGuardrail.class,
                       SecondInputGuardrail.class })
    String chat(String question);

    String doSomethingElse(String question);
}

var assistant = AiServices.create(Assistant.class,
                                  chatModel);
```


182

`MessageModeratorInputGuardrail`

- Package dev.langchain4j.guardrails
- Implémentation d'un `InputGuardrail` fournie par LangChain4j
- Utilise un `ModerationModel`
- La différence avec une simple modération est qu'une requête au modèle de modération est envoyée d'abord, et si ce modèle repère un problème, la requête au LLM n'est pas envoyée (avec la modération simple, les 2 requêtes sont envoyées en //), ce qui convient mieux le plus souvent

183

Exemple

```
ModerationModel = …;
Assistant assistant = AiServices.builder(Assistant.class)
    .chatModel(model)
    .inputGuardrails(
        new MessageModeratorInputGuardrail(moderationModel))
    .build();
String reponse;
try {
  reponse = assistant.chat(question);
} catch(InputGuardrailException e) {
  Throwable cause = e.getCause();
  if (cause instanceof ModerationException){
    reponse = "Quel est votre problème ?" ;
} else {
  throw e;
}
```


184

Tests unitaires guardrails

- Ajouter la dépendance

```
<dependency>
  <groupId>dev.langchain4j</groupId>
  <artifactId>langchain4j-test</artifactId>
  <scope>test</scope>
</dependency>
```

- Cette dépendance fournit des validations supplémentaires pour les tests unitaires

185

Exemple (tutoriel LangChain4j)

```
import static
    dev.langchain4j.test.guardrail.GuardrailAssertions.assertThat;
...
class Tests {
  MyInputGuardrail inputGuardrail = new MyInputGuardrail();
  @Test
  void test() {
    var userMessage = UserMessage.from("Some user message");
    var result = inputGuardrail.validate(userMessage);
    assertThat(result)
        .isSuccessful()
        .hasResult(Result.FATAL)
        .hasFailures()
        .hasSingleFailureWithMessage("Prompt injection detected")
        .assertSingleFailureSatisfied(
           failure -> assertThat(failure)...)
        .withFailures().....
  }
}
```


186

Interface `OutputGuardrail` (1/3)

- La validation est faite après toutes les opérations, y compris les appels d'outils
- Comme `InputGuardrail`, 2 méthodes `default`, dont au moins une doit être implémentée :
  - `OutputGuardrailResult validate(UserMessage userMessage)` : pour les cas simples
  - `OutputGuardrailResult validate(OutputGuardrailRequest params)` : pour les cas où on a aussi besoin d'autres valeurs comme la mémoire, le template du message utilisateur

187

Interface `OutputGuardrail` (2/3)

- Des méthodes `default` pour donner le résultat (type retour `OutputGuardrailResult`) d'un guardrail :
  - `success()` : étape validée
  - `successWith(String)` : étape validée ; en paramètre le texte du succès ; variantes avec paramètres de type `AiMessage`, `String`, `Object`
  - `failure(String, Throwable)` : étape en échec ; en paramètre qui explique le problème ; la cause (au sens des `Exceptions` Java) de l'échec est optionnelle
  - `fatal`, avec les mêmes paramètres que `failure`

188

Interface `OutputGuardrail` (3/3)

- `reprompt(String message, String reprompt)` : le message reprompt est ajouté au contexte du LM et la requête est renvoyée
- `reprompt(String message, Throwable cause, String reprompt)`
- `retry(String)`
- `retry(String, Throwable)`

189

Exemples

- S'assurer que le format de sortie est correct
- Détecter des hallucinations
- Vérifier que la réponse contient bien certaines informations

190

`OutputGuardrailResult` (1/2)

- Classe du package dev.langchain4j.guardrail
- Résultat d'une validation par un `OutputGuardrail`
- La méthode `result()` retourne une instance de l'énumération `Guardrail.Result`
- Quelques autres méthodes :
  - `OutputGuardrailResult blockRetry()` : bloque les retries pour ce résultat
  - `Optional<String> getReprompt()` : retourne le prompt pour réessayer
  - String successfulText() : message du résultat avec succès

191

`OutputGuardrailResult` (2/2)

- Contient des méthodes `static` pour indiquer le résultat :
  - `success()`
  - `successWith(AiMessage)`
  - `successWith(String)`
  - `successWith(AiMessage, Object résultat)`
  - `failure(List<OutputGuardrailResult.Failure>)` : erreur non fatale

192

Déclaration d'un OutputGuardrail

- Par ordre de priorité des classes guardrails sont données :
  1. En paramètres de la méthode `ouputGuardrailClasses` du builder de service IA, ou bien des instances des classes sont passées en paramètres la méthode `outputGuardrails` du builder
  2. En paramètres de l'annotation `@OutputGuardrails` sur une des méthodes qui définit un service IA
  3. En paramètres de l'annotation `@OutputGuardrails` sur l'interface qui définit un service IA

193

Exemple

```
public interface Assistant {
  @OutputGuardrails(
     value = { FirstOutputGuardrail.class,
               SecondOutputGuardrail.class },
     maxRetries = 10
  )
  String chat(String question);
}

var assistant = AiServices.create(Assistant.class,
                                  chatModel);
```


194

`JsonExtractorOutputGuardrail<T>`

- Package dev.langchain4j.guardrails
- Implémentation de `OutputGuardrail` fournie par LangChain4j
- Vérifie qu'une réponse du LM peut être désérialisée de JSON en un objet de type `T`, en utilisant un ObjectMapper de Jackson (https://github.com/FasterXML/jackson-databind)
- Le LM est interrogé à nouveau si ça n'est pas le cas

195

Exemple avec JUnit

```
import static
dev.langchain4j.test.guardrail.GuardrailAssertions.assertThat;
import …
class Tests {
  MyOutputGuardrail outputGuardrail = new MyOutputGuardrail();
  @Test
  void test() {
    var aiMessage = AiMessage.from("Some output");
    var result = outputGuardrail.validate(aiMessage);
    assertThat(result)
      .isSuccessful()
      .hasResult(Result.FATAL)
      .hasFailures()
      .hasSingleFailureWithMessage("Hallucination detected!")
      .hasSingleFailureWithMessageAndReprompt("Hallucination detected!",
           "Please LLM don't hallucinate!")
      .assertSingleFailureSatisfied(failure -> assertThat(failure)...)
      .withFailures().....
    }
}
```

Exemple tutoriel LangChain4j

196

Observabilité

197

Ecouteurs

- LangChain4j a 2 types d'écouteurs :
  - `ChatModelListener` pour surveiller les échanges avec un LM (requêtes, réponses, erreurs)
  - Ecouteurs d'événements pour les services IA

198

Interface ChatModelListener

- Package dev.langchain4j.model.chat.listener
- 3 méthodes `default` :
  - `void onRequest(ChatModelRequestContext requestContext)` : appelée avant l'envoi d'une requête au modèle
  - `void onResponse(ChatModelResponseContext responseContext)` : appelée après la réception d'une réponse envoyée par le modèle
  - `void onError(ChatModelErrorContext errorContext)` : appelée quand une erreur survient pendant l'interaction avec le modèle

199

Exemple (1/2)

```
ChatModelListener listener = new ChatModelListener() {
  @Override
  public void onRequest(ChatModelRequestContext requestContext) {
    ChatRequest chatRequest = requestContext.chatRequest();
    List<ChatMessage> messages = chatRequest.messages();
    System.out.println("Messages : " + messages);
  }

  @Override
  public void onResponse(ChatModelResponseContext
                         responseContext) {
    ChatResponse chatResponse = responseContext.chatResponse();
    AiMessage aiMessage = chatResponse.aiMessage();
    System.out.println("Réponse du LLM : " + aiMessage);
  }
```


200

Exemple (2/2)

```
  @Override
  public void onError(ChatModelErrorContext errorContext) {
    Throwable erreur = errorContext.error();
    erreur.printStackTrace();
  }
};

ChatModel chatModel = OllamaChatModel.builder()
    .baseUrl("http://localhost:11434")
    .modelName("qwen2.5:7b-instruct")
    .listeners(List.of(listener))
    .build();
```


201

Ecouteurs pour service IA

- Pour tracer ce qui se passe durant l'invocation d'un service IA : appels aux LMs, résultats de ces appels, erreurs, ...
- Tous les types d'événement ont un identificateur unique et incluent des informations contenues dans une instance de `InvocationContext`

202

Types d'événements

- `AiServiceStartedEvent` : requête vers un LM
- `AiServiceResponseReceivedEvent` : réponse d'un LM (il peut y en avoir plusieurs pour une requête, par exemple si utilisation d'outils)
- `AiServiceErrorEvent` : échec d'une requête vers un LM
- `AiServiceCompletedEvent` : réussite d'un appel à un LM
- `ToolExecutedEvent` : réussite de l'utilisation d'un outil
- `InputGuardrailExecutedEvent` : intervention d'un guardrail pour une entrée et son issue
- `OutputGuardrailExecutedEvent` : idem pour sortie

203

Ecouteurs d'événements

- Chacun des types peut avoir ses écouteurs
- Par exemple `AiServiceStartedListener`

204

Exemple de code (1/2)

```
public class MonListener implements AiServiceCompletedListener {
  @Override
  public void onEvent(AiServiceCompletedEvent event) {
    InvocationContext context = event.invocationContext();
    Optional<Object> result = event.result();
    // même id pour tous les événements du même appel de LLM
    UUID invocationId = context.invocationId();
    String interfaceName = invocationContext.interfaceName();
    String methodName = invocationContext.methodName();
    List<Object> methodArgs = context.methodArguments();
    Object chatMemoryId = context.chatMemoryId();
    Instant eventTimestamp = context.timestamp();
    // Faire quelque chose avec les données
    }
}
```


205

Exemple de code (2/2)

```
// A la création du service IA :
MonListener monListener = new MonListener();

var monService = AiServices.builder(MonService.class)
      .chatModel(chatModel)  // ou .streamingChatModel(...)
      .registerListener(monListener)
      .build();
```


206

Evénements personnalisés

- Il est possible de créer ses propres événements (implémenter `AiServiceEvent`) et écouteurs (implémenter `AiServiceListener`)
- Pour provoquer un événement, méthode `fireEvent(`*`event`*`)` de `AiServiceListenerRegistrar`

207



Streaming

208

Introduction

- Les modèles génèrent leurs réponses token par token (ou groupe de tokens par groupe de tokens)
- Il est possible de récupérer les tokens dès qu'ils sont générés, plutôt que d'attendre que le modèle ait généré toute sa réponse
- Pour cela, il faut utiliser l'interface `StreamingChatModel`, à la place de `ChatModel`
- Le développeur doit implémenter un handler pour indiquer ce qui se passera quand l'application recevra les tokens



209

StreamingChatModel

- Interface du package dev.langchain4j.model.chat
- 3 méthodes `chat` qui retournent `void` et ont un `StreamingHandler` en dernier paramètre
- Le 1er paramètre représente le ou les messages de la conversation : `ChatRequest`, `String` (pour userMessage) ou `List<ChatMessage>`
- Méthode `void doChat(ChatRequest, StreamingHandler)`
- Méthode `List<ChatModelListener> listeners()`
- Méthode `Set<Capability> supportedCapabilities()`
- Implémentations : `OpenAiStreamingChatModel`, `GoogleAiGeminiStreamingChatModel`, …

210

Streaming avec AI services

- Les AI services permettent de faire simplement du streaming
- Il faut utiliser `TokenStream` comme type retour des méthodes de l'interface de l'assistant IA

211

Interface `TokenStream`

- Package dev.langchain4j.service
- Représente un stream de tokens
- La méthode `start()` démarre l'envoi de la requête au LM et le traitement des tokens émis par le LM en réponse
- On peut recevoir des notifications
  - quand une nouvelle réponse partielle est disponible ; la méthode `onPartialResponse` prend en paramètre comment sera consommée cette réponse (de type `String`)
  - quand le LM a terminé sa réponse ; `onCompleteResponse`) prend en paramètre comment sera consommée la réponse
  - quand une erreur est survenue pendant le streaming (`onError`)
  - ...

212

Exemple

```
TokenStream tokenStream = assistant.chat(question);
tokenStream
  .onPartialResponse(tokens ->
    { /* Traitement quand un groupe de tokens (type
         String) vient d'arriver */ }
  .onCompleteResponse(reponse ->
    { /* Traitement quand la réponse est complète */ }
  .onError(erreur ->
    { /* Traitement de la Throwable */
  .start(); // Démarre traitement de la question et début
            // réception/traitement du stream de tokens
```


213

Exemple de AI service avec streaming et mémoire (1/2)

```
interface Assistant {
  TokenStream chat(String message);
}

// Création d'un assistant :
Assistant assistant = AiServices.builder(Assistant.class)
     .streamingChatModel(modele)
     .chatMemory(chatMemory)
     .build();
```


214

Exemple de AI service avec streaming et mémoire (2/2)

```
TokenStream tokenStream = assistant.chat("...");
CompletableFuture<> future = newCompletableFuture();
tokenStream
  .onPartialResponse(System.out::println)
  .onCompleteResponse(future::complete)
  .onError(future::completeExceptionally)
  .start();
  = future.get(30, SECONDS);
System.out.println("\n" + );
```


215

Méthodes de TokenStream (1/3)

- `TokenStream onPartialResponse(Consumer<String> partialResponseHandler)` appelée à chaque nouvelle réponse partielle disponible
- `TokenStream onCompleteResponse(Consumer<> completeResponseHandler)` appelée quand le LM a fini de répondre (fin du streaming)
- `TokenStream onError(Consumer<Throwable> errorHandler)` appelée quand erreur dans streaming
- `TokenStream onIntermediateResponse( Consumer<ChatResponse> intermediateResponseHandler)`
- `void start()` finit la construction du stream et démarre le streaming

216

Méthodes de TokenStream (2/3)

- `TokenStream onToolExcecuted(Consumer<ToolExecution> toolExecuteHandler)` appelée après qu'un outil a fini son exécution (et avant l'exécution d'un autre outil)
- `TokenStream ignoreErrors()` toutes les erreurs seront ignorées (mais loggées avec le niveau WARN)
- `default TokenStream onRetrieved(Consumer<List<Content>> contentHandler)` appelée quand un Content est retrouvé par un `RetrievalAugmentor`
- `default TokenStream beforeTooleExecution(Consumer<BeforeToolExecution handler)`

217

Méthodes de TokenStream (3/3)

- `default TokenStream onRetrieved(Consumer<List<Content>> contentHandler)` appelée quand un Content est retrouvé par un `RetrievalAugmentor`
- `default TokenStream beforeToolExecution(Consumer<BeforeToolExecution> handler)` appelée juste avant l'exécution d'un outil
- `default TokenStream onPartialThinking(Consumer<PartialThinking> handler)` appelée quand une réponse partielle d'un texte « thinking » est reçu

218

Streaming avec JSF

- Comment faire pour que les tokens envoyés par le LM soient affichés immédiatement et automatiquement sur une page JSF de l'interface utilisateur ?
- 2 solutions :
  - Faire du polling avec JavaScript ; le code JavaScript peut sonder à intervalles réguliers le serveur pour savoir si des nouveaux tokens ont été générés
  - Utiliser un WebSocket

219

APIs et outils autour de IA

220

LlamaIndex

- Comme LangChain, facilite l'intégration des LMs dans des applications, mais avec des approches différentes
- LlamaIndex
  - fournit des outils pour structurer et indexer des données non ou semi-structurées (documents PDF, BDs, APIs,...)
  - s'intègre au RAG pour connecter des LMs à des sources de données externes (avec embeddings ou autres techniques)
  - permet de créer des structures d'index personnalisées pour optimiser la récupération des informations, et d'interroger l'index avec un langage naturel.

221

LangSmith (1/2)

- Plateforme de développement complète pour aider à passer de la phase de prototype à celle de production avec des applications basées sur des LMs
- Offre un ensemble d'outils pour optimiser chaque étape du projet
- Pour débogage, collaboration avec une équipe de développement, tests et surveillance des applications

222

LangSmith (2/2)

- Principales fonctionnalités :
  - Suivi de traces (prompts, réponses LM, contextes)
  - Intégration avec LangChain (version Java en développement https://github.com/langchain-ai/langsmith-java)
  - Expérimentation de différentes configurations de modèles et de prompts pour optimiser les résultats

- Whisper de OpenAI pour « speach to text ». Librairie Java-Whisper. Vidéo YouTube sur cette librairie : https://www.youtube.com/watch?v=ZeH3bBKdqRU. Cette vidéo s'appuie sur le tutoriel de OpenAI https://platform.openai.com/docs/tutorials/meeting-minutes.
- Gradio pour créer des interfaces utilisateur Web pour les modèles de machine learning et de les déployer sans écrire de code

- Extraire la transcription/sous-titres de ce qui est dit dans une vidéo YouTube en utilisant l'API YouTube :
  - https://developers.google.com/youtube?hl=fr pour gérer les vidéos YouTube ; plus particulièrement https://developers.google.com/youtube/v3/docs/captions?hl=fr pour travailler avec les transcriptions
  - Autres possibilités pour travailler avec les transcriptions :
    - Librairie youtube-transcript-api (seulement pour Python) https://pypi.org/project/youtube-transcript-api/
    - Captions grabber https://www.captionsgrabber.com/ un site Web pour travailler avec les sous-titres ; voir vidéo de démonstration https://www.youtube.com/watch?v=OS54TX3YptE

Autres liens intéressants

- https://platform.openai.com/docs/tutorials/web-qa-embeddings. Comment parcourir un site Web pour transformer les pages en « embeddings »

- Projet GitHub https://github.com/kousen/openaidemo en Java 17 pour utiliser Whisper, la génération d'image avec PicoGen, DallE et Stable Diffusion ; voir https://www.youtube.com/watch?v=vRvlqFQGLzQ&list=PLZOgUaAUCiT7ooFAUWId7oeWatv6b3oCD

ElevenLabs

- https://elevenlabs.io/
- Pour « Text to Speech »

Outils pour écrire du code

- https://github.com/jamesmurdza/awesome-ai-devtools, par James Murdza

Type d'outils pour coder

- Complétion de code : GitHub Copilot (gratuit pour universitaires ; https://github.com/features/copilot), Codeium (https://www.codium.ai/ ; version gratuite)
- Assistant pour coder : on peut chater avec un assistant ; vo pour HTML (https://vo.dev/chat)
- Générateur d'interface utilisateur
- Générateur d'applications
- Documentation
- Contrôle de version
- Aide pour les tests

LangChain4j CDI

Présentation

- Projet open source https://langchain4j.github.io/langchain4j-cdi/
- Extension CDI pour faciliter l'intégration de LangChain4j avec Jakarta EE ; Compatible avec Payara, WildFly, Liberty, GlassFish, Quarkus, Helidon
- Permet d'injecter des services IA, des agents et des serveurs MCP
- Offre aussi l'observabilité (telemetry), la tolérance aux pannes, la configuration externe (MicroProfile Config)

Dépendances Maven à ajouter

```
<dependency>
  <groupId>dev.langchain4j.cdi</groupId>
  <artifactId>langchain4j-cdi-build-compatible-ext
  </artifactId>
    <version>${langchain4j-cdi.version}</version>
</dependency>

<dependency>
    <groupId>dev.langchain4j.cdi.mp</groupId>
    <artifactId>langchain4j-cdi-fault-tolerance
    </artifactId>
    <version>${langchain4j-cdi.version}</version>
</dependency>
<!-- Et d'autres pour configuration, serveurs MCP, … -->
```


Exemple

- Définir un service IA :

```
@RegisterAIService(chatModelName = "#default")
public interface ChatBot {
  String chat(String userMessage);
}
```

- L'injecter et l'utiliser :

```
@ApplicationScoped
public class MyBean {
  @Inject
  ChatBot chatBot;
  public String askQuestion(String question) {
    return chatBot.chat(question);
  }
}
```

Références

235

IA avec Java (1/2)

- « AI for Java developers » par Microsoft : https://www.youtube.com/watch?v=V45tKEYYAFs&list=PLPeZXlCR7ew8sdUWqf2itkRG5BUE7GFsy
- TensorFlow pour Java : https://www.baeldung.com/tensorflow-java https://www.tensorflow.org/jvm/install?hl=fr
- Tensorflow et Keras pour Java : https://github.com/dhruvrajan/tensorflow-keras-java Deeplearning4j, https://deeplearning4j.konduit.ai/
- LangChain4j, https://github.com/langchain4j/langchain4j

236

IA avec Java (2/2)

- Playlist YouTube sur IA en Java : https://www.youtube.com/playlist?list=PLZOgUaAUCiT700FAUWId70eWatv6b30CD

237

API d'OpenAI

- OpenAI : https://openai.com
- API de OpenAI : https://platform.openai.com/
- Documentation sur l'API : https://platform.openai.com/docs/api-reference/chat
- Guide pour utiliser l'API de complétion : https://platform.openai.com/docs/guides/text-generation/chat-completions-api
- SDK pour Java : https://github.com/openai/openai-java

238

API de Gemini

- Documentation sur l'API : https://ai.google.dev/gemini-api/docs
- Modèles de l'API : https://ai.google.dev/gemini-api/docs/models
- SDK pour Java : https://github.com/googleapis/java-genai

239

LangChain

- https://www.langchain.com/
- Documentation : https://python.langchain.com/docs/get_started/introduction
- Modules : https://python.langchain.com/docs/how_to/#components
- Quelques articles sur LangChain :
  - https://www.lemondeinformatique.fr/actualites/lire-langchain-un-framework-qui-facilite-le-developpement-autour-des-llm-91921.html
  - https://www.ibm.com/fr-fr/topics/langchain
  - https://aws.amazon.com/fr/what-is/langchain/
  - https://www.lemagit.fr/conseil/Lessentiel-sur-LangChain

240

LangChain4j - liens officiels (1/3)

- https://langchain4j.github.io/langchain4j/
- Documentation : https://docs.langchain4j.dev/
- Javadoc : https://langchain4j.github.io/langchain4j/apidocs/index.html, https://docs.langchain4j.dev/apidocs/index.html
- Code source : https://github.com/langchain4j/langchain4j
- Différentes versions et leurs différences : https://github.com/langchain4j/langchain4j/releases
- Problèmes (*issues*) répertoriés avec LangChain4j : https://github.com/langchain4j/langchain4j/issues

LangChain4j - liens officiels (2/3)

- Tutoriels : https://langchain4j.github.io/langchain4j/tutorials/, https://docs.langchain4j.dev/category/tutorials
- Exemples : https://github.com/langchain4j/langchain4j-examples, https://github.com/langchain4j/langchain4j-examples/tree/main/other-examples/src/main/java
- Get started : https://docs.langchain4j.dev/get-started
- Discussions : https://github.com/langchain4j/langchain4j/discussions
- Q&A : https://github.com/langchain4j/langchain4j/discussions/categories/q-a

LangChain4j - liens officiels (3/3)

- Intégration Gemini : https://github.com/langchain4j/langchain4j/blob/main/docs/docs/integrations/language-models/google-ai-gemini.md
- Intégration OpenAI : https://github.com/langchain4j/langchain4j/blob/main/docs/docs/integrations/language-models/open-ai-official.md

Anciennes versions de LangChain4j

- Pour la javadoc, aller sur https://javadoc.io/doc/dev.langchain4j/langchain4j-core et choisir la version en haut
- Pour le reste, aller sur le dépôt Maven Central ; par exemple https://mvnrepository.com/artifact/dev.langchain4j/langchain4j/0.36.0 ; en cliquant sur « View All » de la ligne Files, on peut récupérer, par exemple, la javadoc ou le source

Code source

- Le code est constitué de modules
  - langchain4j est le module principal
  - langchain4j-core contient les classes de base : Document, TextSegment, ...
  - de nombreux modules dédiés aux différents produits supportés par LangChain4j : langchain4j-openai, langchain4j-google-ai-gemini, langchain4j-hugging-face, ...
- Lombok était utilisé au début de LangChain4j mais il en cours de suppression ; du code ancien utilise encore Lombok et des getters et setters n'apparaissent alors ni dans le code, ni dans la javadoc

LangChain4j (1/3)

- https://www.youtube.com/watch?v=cjI_6Siry-s
- https://www.youtube.com/watch?v=Ewr1KYPtLao
- AI services : https://www.sivalabs.in/langchain4j-ai-services-tutorial/, https://docs.langchain4j.dev/tutorials/ai-services
- Generative AI Conversations using LangChain4j ChatMemory : https://www.sivalabs.in/generative-ai-conversations-using-langchain4j-chat-memory/
- Getting Started with Generative AI using Java, LangChain4j, OpenAI and Ollama : https://www.sivalabs.in/getting-started-with-generative-ai-using-java-langchain4j-openai-ollama/

LangChain4j (2/3)

- LangChain4j. Webinar organisé par Arun Gupta, avec Marcus Hellberg ; à la fin montre comment on peut charger un contexte personnalisé pour que le bot en tienne compte dans sa démo : https://twitter.com/i/broadcasts/1yNxaZyPzXRKj
- Avec Quarkus, passe un code HTML à l'API OpenAI et résume le contenu du HTML ; le code HTML est passé en morceaux : https://developers.redhat.com/articles/2024/02/07/how-use-llms-java-langchain4j-and-quarkus#exploring_the_capabilities_of_langchain4j_and_quarkus

LangChain4j (3/3)

- https://kindgeek.com/blog/post/experiments-with-langchain4j-or-java-way-to-llm-powered-applications : article très complet sur LangChain4j (6/2/24)
- Vidéo complète sur les outils (@Tool) : https://youtu.be/cjI_6Siry-s?si=nAk9AjK2dajT2b63
- Vidéo de presque 3 heures sur LangChain4j : https://www.youtube.com/watch?v=jzuP6l54kWA
- https://javapro.io/2025/04/23/build-ai-apps-and-agents-in-java-hands-on-with-langchain4j/

LangChain4j et réseaux sociaux

- Stackoverflow : https://stackoverflow.com/questions/tagged/langchain4j
- Discord : https://discord.com/channels/1156626270772269217/1156626271212666882

LangChain4j et LMs

- Particularité des LMs : https://docs.langchain4j.dev/category/language-models/
- Pour Gemini : https://docs.langchain4j.dev/integrations/language-models/google-ai-gemini
- Pour OpenAI : https://docs.langchain4j.dev/integrations/language-models/open-ai

MOOCs Udemy sur LangChain

- https://www.udemy.com/course/langchain-in-action-develop-llm-powered-applications/
- https://www.udemy.com/course/langchain-python-french/ ; bonne présentation gratuite (environ 1 heure), traduction automatique en français de Melt Labs d'un cours en anglais ; Python

MOOCs et vidéos sur LangChain

- La playlist de courtes vidéos sur les LMs, ChatGPT, LangChain : https://www.youtube.com/playlist?list=PLKWoAUxdZEb_BqGgm-Rk7fiPUXccFHBGB
- https://www.youtube.com/watch?v=uJJ6uP5IViA
- https://www.youtube.com/watch?v=1VeYoNoXIgU

- Série de 4 courtes vidéos qui montrent comment utiliser Hugging Face et Ollama pour faire du fine-tuning avec Llama-3.2
  - https://www.youtube.com/watch?v=RAubwMSPRTo
  - https://www.youtube.com/watch?v=wco_8l_zh7s
  - https://www.youtube.com/watch?v=VePkG2EQKIM
- Semantic Kernel : kit de développement open source pour faciliter l'utilisation de l'IA en Java, Python, C# https://learn.microsoft.com/en-us/semantic-kernel/overview/

253

Projets avec LangChain4j

- https://github.com/langchain4j/awesome-langchain4j

254